

HAUSARBEIT  
Leonhard Lüdeke

# Ein KI-Gamebot für Vier Gewinnt. Implementiert in Erlang

---

FAKULTÄT INFORMATIK UND DIGITALE GESELLSCHAFT

Faculty of Computer Science and Digital Society

Betreuung durch: Prof. Dr. Christoph Klause  
Eingereicht am: 23. Januar 2026

---

HOCHSCHULE FÜR ANGEWANDTE  
WISSENSCHAFTEN HAMBURG  
Hamburg University of Applied Sciences

## Inhaltsverzeichnis

|          |                                       |           |
|----------|---------------------------------------|-----------|
| <b>1</b> | <b>Einleitung</b>                     | <b>2</b>  |
| 1.1      | Das Spiel Vier Gewinnt . . . . .      | 2         |
| <b>2</b> | <b>Lösungsansätze</b>                 | <b>3</b>  |
| 2.1      | Minimax-Algorithmus . . . . .         | 3         |
| 2.2      | Alpha-Beta Optimierung . . . . .      | 5         |
| 2.3      | Monte-Carlo-Treesearch . . . . .      | 6         |
| 2.4      | Wahl des Algorithmus . . . . .        | 7         |
| 2.5      | Die Bewertungsfunktion . . . . .      | 8         |
| <b>3</b> | <b>Entwurf</b>                        | <b>9</b>  |
| 3.1      | Spielbrett . . . . .                  | 9         |
| 3.2      | Gamebot . . . . .                     | 10        |
| 3.2.1    | Suchreduzierung . . . . .             | 10        |
| 3.2.2    | Heuristik . . . . .                   | 11        |
| 3.2.3    | Zugberechnung . . . . .               | 11        |
| <b>4</b> | <b>Realisierung des Entwurfs</b>      | <b>13</b> |
| 4.1      | Spielbrett . . . . .                  | 13        |
| 4.2      | Gamebot . . . . .                     | 15        |
| 4.2.1    | Entwurfsänderung . . . . .            | 15        |
| 4.2.2    | Konstanten statt Strukturen . . . . . | 15        |
| 4.2.3    | Ablauf der Hauptschleife . . . . .    | 16        |
| <b>5</b> | <b>Analyse</b>                        | <b>17</b> |
| 5.1      | Tournaments . . . . .                 | 19        |
| 5.1.1    | 1. Versuch . . . . .                  | 19        |
| 5.1.2    | 2. Versuch . . . . .                  | 20        |
| 5.1.3    | 3. Versuch . . . . .                  | 21        |
| <b>6</b> | <b>Fazit</b>                          | <b>22</b> |
| <b>7</b> | <b>Anhang</b>                         | <b>23</b> |
| 7.1      | Datenstrukturen . . . . .             | 23        |
| 7.1.1    | Wichtigsten Funktionen . . . . .      | 23        |

|                                    |           |
|------------------------------------|-----------|
| <b>Literatur</b>                   | <b>27</b> |
| <b>Selbstständigkeitserklärung</b> | <b>28</b> |

Diese Hausarbeit, die im Rahmen des Moduls Algorithmen und Datenstrukturen entstanden ist, beschäftigt sich mit dem Entwurf eines Gamebots für das Spiel Vier Gewinnt. Zu Beginn werden das Spiel und seine Regeln erklärt sowie bekannte Ansätze beleuchtet, die sich als Gamebot für Vier Gewinnt eignen. Anschließend wird ein Entwurfsmuster entwickelt, welches den ausgewählten Algorithmus (Gamebot) mit den gegebenen Codevorgaben und Board Schnittstellen darstellt. Darauf folgt die Beschreibung der Implementierung des Entwurfes, und eine Analyse des erstellten Codes.

**Keywords: Vier Gewinnt, KI-Gamebot, Erlang**

# 1 Einleitung

## 1.1 Das Spiel Vier Gewinnt

Vier Gewinnt ist ein klassisches Strategiespiel, und wurde 1973 von Howard Wexler entwickelt und ist ein endliches Zwei-Personen Nullsummenspiel mit perfekter Information. Ein Nullsummenspiel kennzeichnet sich dadurch aus, dass die Verluste des/der einen Spielers/in exakt den Gewinnen des/der anderen darstellen. Eine spielende Person ist deshalb perfekt informiert, da die gesamte Spielsituation jederzeit offen einsehbar ist. Somit kann ein/e Spieler/in eine voll informierte Entscheidung treffen. [7]

Das Spiel wird auf einem vertikal aufgerichteten Brett mit sieben Spalten und sechs Zeilen gespielt (7x6). Die Spielenden werfen abwechselnd Steine ihrer Farbe von oben in eine Spalte ein, wobei der Stein auf die unterste freie Position fällt. Dabei stehen jeder Person 21 Spielsteine zur Verfügung. Gewinner ist diejenige Person, die zuerst vier eigene Steine horizontal, vertikal oder diagonal in einer Linie anordnet. [9]

Die Kombination aus einfacher Spielmechanik und überraschend hohen Gesamtzahl der legalen Spielpositionen von circa 4,5 Billionen [3], während Tic-Tac-Toe 5.478 gültige Positionen besitzt, macht dieses Spiel zu einem Interessanten Untersuchungsobjekt. Der Aufbau eines Kompletten Spielbaumes ist, mit  $4 \times 10^{21}$  Blattknoten, aufwendig. Zusätzlich ist die Laufzeit, für die Berechnung des optimalen Spielzugs, in diesem Spielbaum praktisch unbrauchbar, wenn auch potenziell perfekt. Das ermitteln des Perfekten Zuges, für einen Gamebot, liegt somit nicht in einer 1 zu 1 Abbildung aller Spielzustände, sondern in Algorithmen Ansätzen, welche ich im nächsten Unterkapitel beleuchtet werden.

An dieser Stelle sei erwähnt, dass Victor Allis [2] und zeitgleich auch James D. Allen [2] 1988 das Spiel vollständig lösten. Beide sind zu dem Ergebnis gekommen, dass die beginnende Person, bei perfektem Spielverhalten, immer gewinnt, wenn sie den ersten Stein in die mittlere Spalte einwirft. Wenn nicht, endet das Spiel mit einem Unentschieden.

## 2 Lösungsansätze

Wie oben beschrieben ist es, aufgrund des großen Suchraumes, unpraktikabel alle Spielstände abzubilden. Der Unterschied, der einzelnen Lösungsansätze, liegt darin wie dieser Zustandsraum durchsucht und der Suchbaum aufgebaut wird. Alle Verfahren haben dabei gemein, dass sie den Zustandsraum mit Knoten (Stellungen im Spiel) und Kanten (ein Zug) darstellen. Nachfolgend sind Knoten und Kanten als diese Zustände zu verstehen. Je nachdem welcher Algorithmus beschrieben wird, können diese Zustände zusätzliche Eigenschaften bekommen.

### 2.1 Minimax-Algorithmus

Ein fundamentales Konzept zur Berechnung von optimalen Strategien in Nullsummenspielen ist der Minimax-Algorithmus [8]. Dieser Algorithmus basiert auf einem Spielbaum, in dem jeder Knoten ein MAX- oder MIN-Knoten ist. Diese geben, neben dem Spielzustand, an welcher Person (Maxi- oder Minimierer/in) am Zug ist und besitzen einen numerischen Wert. Dieser numerische Wert gibt an welcher Spielzustand besser für die jeweilige Person ist. Der sogenannte Maximierer/in (meist 1. Spieler/in) wählt immer den Spielzustand mit dem höheren Wert und der Minimierer/in (meist 2. Spieler/in), den kleineren Wert. Dies entspricht einer Optimalen Spielweise.

Die Werte haben ihren Ursprung in den Blattknoten (Endzustände) des Baumes, welche zeitgleich einen Terminalen Zustand, also das Ende eines Spiels darstellen. Diese besitzen eine heuristische Bewertungsfunktion, die aussagt wie gut oder schlecht dieser Endzustand für die Maximierende Person ist. Die Werte für die Inneren Knoten setzen sich aus Ihren Kindknoten zusammen, also durch das Propagieren der Werte Ihrer Kindknoten.. Der Wert an der Wurzel gibt somit den Wert des terminalen Zustands an, welcher beim Perfektem Spielverhalten beider Spieler erreicht wird. Die Abbildung 1 zeigt diese Variante und hebt den Pfad in rot hervor. Dieser Pfad wird Princibal-Variation genannt.

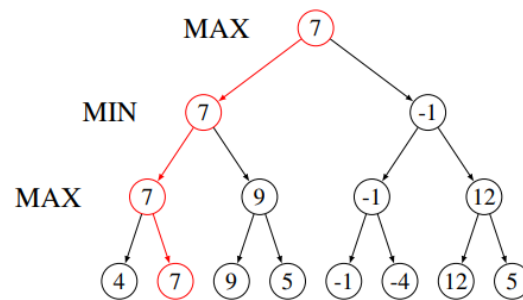


Abbildung 1: Abbildung eines Minimax-Spielbaumes [6]

Klassisch wird dieser Algorithmus mittels einer Tiefensuche implementiert und besitzt ohne Optimierung eine exponentielle Laufzeit von  $O(b^d)$  wobei  $b = \text{mögliche Spalten pro Zug}$  und  $d = \text{Züge im Vorraus}$  darstellen. Dabei werden zunächst die Minimax-Werte aller möglichen Nachfolgeknoten eines Knotens berechnet. Ist der aktuelle Knoten ein MAX-Knoten, übernimmt er den höchsten Minimax-Wert seiner Kindknoten. Handelt es sich hingegen um einen MIN-Knoten, erhält er den niedrigsten Minimax-Wert aus seinen Kindknoten. Somit lässt sich der Algorithmus aus zwei Teilen zusammensetzen.

Einer Funktion, die der minimierende Spieler verwendet, und einer, die vom maximierenden Spieler aufgerufen wird. Abhängig davon, ob der aktuell betrachtete Knoten ein MIN- oder MAX-Knoten ist, übernimmt dieser den Wert des Kindknotens mit dem jeweils kleinsten oder größten Wert. Dieses Vorgehen lässt sich jedoch zu einer einzigen Komponente vereinheitlichen, indem man das Minimierungsproblem des MIN-Spielers in ein Maximierungsproblem umwandelt. Dazu wird die Bewertungsfunktion des minimierenden Spielers einfach negiert, sodass beide Spieler effektiv ein Maximierungsproblem lösen. Diese vereinfachte Form ist als *Negamax-Verfahren* bekannt. Nachfolgend ein Beispiel in Pseudocode:

```
negamax(Knoten) :
    wenn Knoten ein Blatt ist:
        gib den Wert des Knotens zurück
    Wert = -minimalerNegativWert
    für alle Kinder von Knoten:
        Wert = max(Wert, -negamax(Kind))
    gib Wert zurück
```

## 2.2 Alpha-Beta Optimierung

Die Alpha-Beta-Pruning-Technik ist eine Optimierungsmethode des Minimax-Algorithmus, die durch intelligentes Abschneiden von Spielbaumästen die Anzahl der zu evaluierenden Knoten erheblich reduziert [4]. Das Verfahren arbeitet mit zwei Werten:

- **Alpha** ( $\alpha$ ): Die beste (höchste) Bewertung, die der Maximierer bisher gefunden hat.
- **Beta** ( $\beta$ ): Die beste (niedrigste) Bewertung, die der Minimierer bisher gefunden hat.

Dabei stellte *Knuth* [4] in seiner Analyse eine obere und untere Wertschranke dar, woraus sich folgendes Suchfenster ergibt:

$$\alpha \leq \text{MinimaxWert} \leq \beta$$

Ein Ast kann demnach abgeschnitten werden, wenn ein besserer Minimax-Wert außerhalb des Suchfensters gefunden wird, da die restlichen Züge in diesem Ast nicht mehr relevant für die optimale Spielentscheidung sind. Dies basiert auf der Annahme, dass beide Spieler optimal spielen und keine schlechteren Positionen akzeptieren würden [4].

In der Regel werden  $\alpha$  und  $\beta$  auf den schlechtesten Wert für den jeweiligen Spieler gesetzt. Für  $\alpha = -\infty$  und für  $\beta = \infty$ . Diese werden aktualisiert und die jeweiligen Knoten aktualisiert und auf die Bedingung des Suchfensters geprüft. In der Abbildung 2 ist zu sehen wie sich der Suchbaum reduziert. Um nicht zwei verschiedene Algorithmen implementieren zu müssen kann die vorhin beschriebene *Negamax* variante angewandt werden. Hier ein Beispiel als Pseudocode:

```
AlphaBetaNegamax(Knoten, Alpha, Beta):  
    wenn der Knoten ein Blatt ist:  
        gib den Wert des Knotens zurück  
    maxWert = -maximalNegativWert  
    für jeden Nachfolger des Knotens:  
        Wert = -AlphaBetaNegamax(Nachfolger, -Beta, -Alpha)  
        maxWert = max(maxWert, Wert)  
    Alpha = max(Alpha, Wert)  
    wenn Alpha >= Beta:  
        breche ab  
    gib maxWert zurück
```

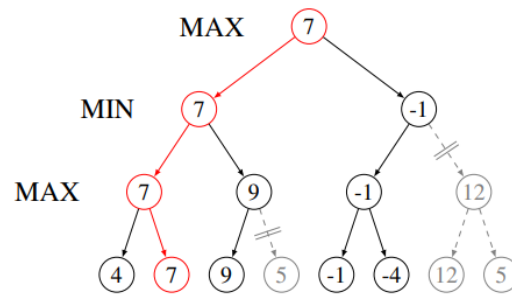


Abbildung 2: Abbildung eines Minimax-Spielbaumes [6]

Die Effektivität von Alpha-Beta-Pruning hängt stark von der Reihenfolge ab, in der die Züge evaluiert werden und wie gut die Bewertungsfunktion mit den Terminalen zuständen umgeht. Mit optimaler Zugreihenfolge ist eine Reduktion der Komplexität auf  $O(b^{d/2})$  möglich, was eine quadratische Verbesserung gegenüber dem klassischen Minimax darstellt [4].

### 2.3 Monte-Carlo-Treesearch

Ein alternatives Konzept zur Berechnung von annähernd optimalen Strategien in Nullsummenspielen ist die Monte-Carlo-Baumsuche (MCTS). Wie der Minimax-Algorithmus basiert auch MCTS auf einem Spielbaum, in dem jeder Knoten einen Spielzustand repräsentiert und einen numerischen Wert besitzt. Im Gegensatz zum Minimax-Algorithmus, der durch statische Bewertungsfunktionen an den Blattknoten operiert, bestimmt MCTS diese Werte durch wiederholte stochastische Simulationen (Rollouts) bis zu den Endzuständen des Spielbaums. Durch die Propagierung der Simulationsergebnisse von den Blattknoten zu inneren Knoten konvergiert die Schätzung der Knotenwerte mit hinreichend vielen Iterationen gegen gute Näherungen der optimalen Spielqualität [5].

Die MCTS-Methode durchläuft zur Wertberechnung vier konzeptionelle Phasen: In der Selection-Phase werden bestehende Knoten gemäß der UCB-Heuristik ausgewählt, die Exploration und Exploitation balanciert. Die Expansion-Phase fügt neue Knoten hinzu, während die Simulation-Phase Zufallsspielzüge bis zum Endzustand durchführt. In der abschließenden Backpropagation-Phase werden die Ergebnisse dieser Simulationen entlang des durchlaufenen Pfades rückwärts propagiert und aktualisieren die Knotenwerte sowie Besuchszahlen.

Diese Methode wird iterativ implementiert und benötigt, im Gegensatz zum klassischen Minimax mit Tiefensuche, keine explizite Bewertungsfunktion. Während Minimax ohne Optimierung exponentielle Laufzeit von  $O(b^d)$  aufweist, wobei  $b = \text{mögliche Spalten pro Zug}$  und  $d = \text{Züge im Voraus}$ , funktioniert MCTS praktikabler bei Spielen mit moderater Komplexität wie 4 Gewinnt. Mit nur 32 MB Speicher und etwa 500.000 Simulationen erreicht MCTS zuverlässige Spielstärke. Damit bietet MCTS eine effiziente Alternative zu Alpha-Beta Pruning, insbesondere wenn keine domänenspezifische Evaluierungsfunktion vorhanden ist.

### 2.4 Wahl des Algorithmus

Für diese Implementierung habe ich mich für den **Minimax-Algorithmus mit Alpha-Beta-Pruning** entschieden [8, 4]. Diese Wahl wird durch folgende Faktoren begründet:

1. **Vorhersagbarkeit und Optimalität:** Minimax mit Alpha-Beta-Pruning garantiert die Findung des optimalen Zuges unter Annahme optimalen gegnerischen Spiels, sofern die Suchtiefe ausreichend ist [8].
2. **Effiziente Komplexitätsreduktion:** Mit Alpha-Beta-Pruning kann die Komplexität quadratisch reduziert werden, was tiefere Suchbäume ermöglicht. [4].
3. **Persönliche Präferenz:** Zu guter Letzt fiel die Wahl ebenfalls auf eine Implementierung mittels des Minimax Algorithmus, da ich den MCTS schon einmal implementiert habe.

### 2.5 Die Bewertungsfunktion

Nach der näheren Betrachtung in der Section zu Minimax 2.1 und Alpha-Beta 2.2 wird klar das die Bewertungsfunktion ein zentrales Element dieses Algorithmuses ist. Daher stellt sich die Frage wie diese Bewertungsfunktion sich genau bildet.

Die Zentrale frage ist wie die einzelnen Spielzustände bewertet werden um deren Bewertung für Bewertungsfunktion zu nutzen. Hierfür gibt es zum Beispiel die Möglichkeit jedes Feld eine Bewertung zu geben. Diesen Ansatz verfolgt Liese, Yassin [6] in ihrer Bachelorarbeit, indem sie ein Framework zur Optimierung Minimax-basierter Suchstrategien anhand des Spiels Vier-Gewinnt entwickelt. Jedoch gibt es einen sehr bewehrten Ansatz der 4-Fenster-Bewertung.

Dabei wird ein Spielzustand in alle möglichen 4-Fenster (horizontal, vertikal und diagonal) zerlegt und jede Konstellation gewichtet. Es gibt eine generelle Evaluierungsfunktion die wie folgt aussieht.

$$Eval = \sum_{i=1}^n w_i f_i(s)$$

Mittels der Funktionen  $f_i()$  werden aus dem Aktuellen Spielzustand  $s$  Eigenschaften ermittelt. Diese können zum Beispiel Anzahl der eigenen Steine in der Mitte des Spielfelds sein (meistens werden die Besser bewertet), der vorhin erwähnte 4-Fenster-Bewertung oder auch allgemein Adjazente Steine. Wichtig ist das mittels der Gewichte  $w_i$  die jeweiligen Ergebnisse der Funktionen Gewichtet werden und legen dadurch fest wie viel Relevanz eine Eigenschaft (Funktion) erhält. Des weiteren werden auch Potenzielle Gewinn oder Verlust Situtation häufig extream Stark bewertet, sodass ein Gewinn Zug nicht übersehen wird. Einige dieser Ansätze sind in einem Project namens "roboticsproject"[1] sehr ausführlich beschrieben.

## 3 Entwurf

### 3.1 Spielbrett

Das Spielbrett ist Zentral für das Spiel 4 Gewinnt und wurde vom Professor Dr. Klauck vorgegeben. Die Schnittstellendefinition sieht folgende Operatoren vor:

- new\_board/0:  $\rightarrow$  board
- is\_board/1: board  $\rightarrow$  boolean
- export\_board/1: board  $\rightarrow$  boardMatrix
- print\_board/1: board | boardMatrix  $\rightarrow$  done
- next\_player/1: board | {<blue | red>, <count>}  $\rightarrow$  {<red | blue>, <count>+1}
- current\_player/1: board  $\rightarrow$  {<red | blue>, <count>}
- score/1: board  $\rightarrow$  <running | red | blue | remis>
- possible\_moves/1: board  $\rightarrow$  number-list
- possible\_moves\_pos/1: board  $\rightarrow$  number-list
- make\_move/2: board  $\times$  number  $\rightarrow$  board | {error, {game\_over, <score>}}  
| {error, column\_full}  
| {error, column\_outofrange}
- score\_ChipsRow/1: board  $\rightarrow$  number-list
- score\_ChipsRow/2: board  $\times$  {<red | blue>, <count>}  $\rightarrow$  number-list
- check\_geq\_three/2: board  $\times$  {<red | blue>, <count>}  $\rightarrow$  number-list

Auf Basis dieser Operatoren habe ich die Datenstrukturen und zusätzliche Funktionen überlegt die bei der Schnittstellen Realisierung unterstützen. Einfache Getter und potenzielle Hilfsfunktionen sind nicht aufgelistet. Die nähere Beschreibung finden Sie im Anhang (7)

## 3.2 Gamebot

Um mit einem Gamebot für 4 Gewinnt zu starten bedarf es einer Struktur die einen Knoten in einem Suchbaum darstellen kann und einen Spielzustand repräsentiert. Hierzu kann eine Struktur erstellt werden die wie folgt aussieht:

```
Node = {  
  board: Board,           // Spielzustand  
  value: Integer,         // Heuristischer Wert  
  depth: Integer,         // Tiefe im Baum  
  move: Integer,          // Spalte, die zu diesem Zustand führte  
  is_terminal: Boolean    // Ist Endzustand?  
}
```

Der Gamebot nutzt den in Kapitel 2.1 beschriebenen Minimax-Ansatz, wobei die negaMax variante umgesetzt wird. Um die zu evaluierenden Knoten zu reduzieren wird Alpha-Beta-Pruning eingesetzt, welches in Kapitel 2.2 ebenfalls mit einem Pseudocode beschrieben wird.

### 3.2.1 Suchreduzierung

Für die Suchreduzierung wird ein neue Struktur eingeführt über welche sich die Wertschranken steuern lassen. In der Regel werden alpha und beta extrem hoch angesetzt damit Terminale im Baum sehr gut erkannt werden. Diese Struktur sieht wie folgt aus:

```
AlphaBeta = {  
  alpha: Integer,         // -? initial (z.B. -1000000)  
  beta: Integer           // +? initial (z.B. 1000000)  
}
```

#### 3.2.2 Bewertungsfunktion

Wie im Kapitel 2.5 zur Bewertungsfunktion bedarf es Gewichte um die jeweilige Spielsituation zu Bewerten. Dafür eignet es sich ebenfalls eine Struktur anzulegen und so aussehen kann:

```
EvalWeights = {  
    win: Integer,          // Gewinn (z.B. 10000)  
    lose: Integer,         // Verlust (z.B. -10000)  
    three_in_row: Integer, // Drei in einer Reihe (z.B. 100)  
    two_in_row: Integer,   // Zwei in einer Reihe (z.B. 10)  
    center_column: Integer // Bonus für mittlere Spalten (z.B. 5)  
}
```

Das ermöglicht bei Bedarf eine Anpassung der Gewichte ohne den Suchalgorithmus anpassen zu müssen. Des weiteren beschrieb ich in die Methode des 4-Fenster Scores welche ich einsetzen möchte um die Heuristik genauer zu machen. Als Hilfsstruktur bietet sich eine 4er Liste an, da diese genau diesen Zustand repräsentiert:

```
Window = [Cell, Cell, Cell, Cell]
```

Der Center Bonus ist ebenfalls ein gängiges Mittel da die Mittleren Felder mehr Möglichkeiten eröffnen 4 Steine in einer Reihe zu bekommen. Ebenfalls sollte eine Zugreihenfolge berücksichtigt werden, da diese einen Starken Einfluss auf die Effektivität des Algorithmuses hat, wie oben im Theorie teil beschrieben.

#### 3.2.3 Zugberechnung

Nachdem Blick auf die wichtigsten Hilfsstrukturen möchte ich nun einen Blick auf den Algorithmischen Ablauf des Gamebots legen. In dem Ablaufdiagramm (7.1.1) sind verwendete Methoden aus der Schnittstelle des Boards beschrieben, sowie Methodennamen für den KI bot vergeben. Die Einzelnen Schritte sind mit Nummern und Kleinbuchstaben versehen um diese Stellen im Code wieder zu finden.

Generell ist der Ablauf wie folgt.

- 0-0a Spielstatus prüfen | score = running?
- 1 Aktuellen Spieler holen
- 2-2a mögliche Züge ermitteln
- 3 Züge nach Relevanz Sortieren
- 4 Über moves laufen | Wenn fertig  $\rightarrow$  5.
- 4-4d Negamax Suche mit Alpha-Beta Optimierung, für jede Spalte in moves
- 4e  $\text{Alpha} \geq \text{Beta}$  | Ja  $\rightarrow$  5 - Nein  $\rightarrow$  zur nächsten col (4)
- 5 BestMove gesetzt bzw. gefunden | Ja  $\rightarrow$  6. - Nein  $\rightarrow$  return error
- 6 makeMove und gebe neues Board zurück.

Zu Punkt Nummer 3 in denen Züge nach Relevanz Sortiert werden ergibt es Sinn mittlere Spalten zu bevorzugen, da diese Statistisch bessere Positionen sind. Die Negamax Suche und Alpha-Beta Optimierung wurde eingehend beschrieben und muss für jede potenzielle neue Spielposition ausgeführt werden.

```
l> competition:oneGame().
empty empty empty empty empty empty empty
empty empty empty empty empty empty empty
empty empty empty empty empty empty empty
empty empty empty empty empty empty empty
empty empty empty empty empty empty empty
empty empty empty empty empty empty empty

Bitte wählen Sie aus der Liste der möglichen Spalten: [1,2,3,4,5,6,7]
4
Sie haben 4 gewählt.
empty empty empty empty empty empty empty
empty empty empty empty empty empty empty
empty empty empty empty empty empty empty
empty empty empty empty empty empty empty
empty empty empty B empty empty empty
empty empty empty R empty empty empty

Bitte wählen Sie aus der Liste der möglichen Spalten: [1,2,3,4,5,6,7]
Geben Sie Ihre Auswahl ein: Command is taking a long time, type Ctrl+G.
5
Sie haben 5 gewählt.
empty empty empty empty empty empty empty
empty empty empty empty empty empty empty
empty empty empty empty empty empty empty
empty empty empty B empty empty empty
empty empty empty R empty empty empty
empty empty empty B R empty empty

Bitte wählen Sie aus der Liste der möglichen Spalten: [1,2,3,4,5,6,7]
```

Abbildung 3: Screenshot fehler in put\_in\_col/3

## 4 Realisierung des Entwurfs

### 4.1 Spielbrett

Der Entwurf definiert das Board (7.1) als 4er Tupel, mit einer Matrix als Liste von 7 Spalten und einer Spalte als Liste von 6 Zellen. Diese Struktur wird mithilfe der Initialisierungsfunktion `new_board()` 7.1.1 und den Hilfsfunktionen `make_empty_board/2` und `make_col/2`.

Die Umsetzung der eigentlichen Züge und dem damit verbundenen Spielerwechsel wird mit `make_move/2` 7.1.1 vollzogen. Dabei wird die Prüfung auf den Score (ob das Spiel noch läuft (running)) in der Methode vollzogen und der eigentliche Spielzug in `do_move/3`. Nach einer Index Überprüfung führt die `place_chip` Funktion das tatsächliche Fallenlassen aus. Hier wird eine weitere Hilfsfunktion namens `put_in_col/3` genutzt. Diese habe ich vorerst falsch Implementiert wie in der obigen Abbildung 4.1 zu erkennen.

Dabei stellte ich fest, dass die Implementierung der Accumulator welcher die belegten Felder einsammelt, wieder reversioniert (zurückgesetzt) werden muss, damit die Tabelle korrekt aufgebaut wird.

Ein weiteres wichtiges Element ist die Score Berechnung. Die Funktionen `score/1` und `calc_score/1` aus dem Entwurf 7.1.1 setzen dies um. Wobei eine Hilfsfunktion in

`calc_score/1` namens `winner_check/2`. Dieser ruft 4 Funktionen auf die jeweils prüfen ob eine Farbe Horizontal/Vertikal, Diagonal von links nach unten rechts (`check_diag1`) oder Diagonal von links unten nach rechts (`check_diag2`) oben gewonnen hat. Um die Effizienz der Prüfung zu steigern wird mit der Funktion `cols_to_rows/1` die Matrix in eine Zeilenansicht überführt. Das erleichtert die Prüfungen, weil die Zeilenbasiert intuitiver ist und es die Bildschirmausgabe erleichtert. Außerdem reduziert dies die Zugriffe in der Horizontalen Prüfung, da nur eine Liste überprüft werden muss.

Alle nötigen Iterationen werden rekursiv durchgeführt und das Pattern Matching angewandt. Einige Operationen für Listen die sonst aus dem In Build Modul List kommen sind hinzugefügt worden. Diese erleichtern das Arbeiten mit Listen enorm.

- `append(A, B)` – verkettet Listen A und B (`A ++ B / lists:append(A, B)`)
- `get_nth(Liste, N)` – gibt N-tes Element (1-basiert) aus Liste zurück
- `len_check(Liste, ErwarteteLaenge)` – prüft ob Liste exakt die gegebene Länge hat
- `reverse(Liste, Akku)` – kehrt Liste um

Zu guter Letzt sichert die Funktion `is_board/1` ab dass das Board die korrekte Struktur besitzt, sodass `export_board/1` und `make_move/2` korrekt auf dem Board arbeiten können.

### 4.2 Gamebot

#### 4.2.1 Entwurfsänderung

Die Implementierung des Board hat dazu geführt das ich den Entwurf des Gamebots überdacht habe. Denn die Informationen, welche ich in dem Node 3.2 halte sind teilweise schon in dem Board vorhanden oder können von diesem Abgefragt werden. Hier sei besonders die Terminal Logik erwähnt, also wann ein Endzustand erreicht ist.

Somit bietet das Board ausreichend Schnittstellen um Informationen als Funktionsparameter weiter zu geben, weshalb der Entwurf angepasst wurde. Das Ablaufdiagramm bleibt jedoch erhalten.

#### 4.2.2 Konstanten statt Strukturen

Die beschriebenen Strukturen aus dem Kapitel 3.2 werden umgewandelt in Konstanten und beziehen jetzt auch Gewisse Situationen mit ein. Ist also ein WIN in Sicht wird dieser Zug sehr positiv gewertet und ein LOSE dementsprechend negativ. Ebenfalls kommt ein Bonus für eine Zentrale Spalte hinzu, sowie Boni für 3 oder 2 aneinanderliegende Steine.

```
-define(WIN, 10000) .  
-define(LOSE, -10000) .  
-define(THREE_IN_ROW, 100) .  
-define(TWO_IN_ROW, 10) .  
-define(CENTER_BONUS, 5) .  
-define(INITIAL_ALPHA, -1000000) .  
-define(INITIAL_BETA, 1000000) .  
-define(SEARCH_DEPTH, 6) .  
-define(BOARD_WIDTH, 7) .  
-define(BOARD_HEIGHT, 6) .
```

### 4.2.3 Ablauf der Hauptschleife

Da die Züge wie im Ablaufdiagramm 7.1.1 im Punkt 4, nacheinander ausgeführt werden, und so ein neues Board entsteht, kann mithilfe der Schnittstellen des Board (vor allem `score/1`, `possible_moves/1`, `make_move/2`, `export_board/1`) nach und nach rekursiv die Möglichen Stellungen abgegangen und bewertet werden.

Die Hauptschleife des Bots beginnt mit der Funktion `find_best_move/6`, Punkt 4 im Ablaufdiagramm. Dabei wird für jede Spalte (jeden möglichen Zug) ein neues Board generiert (mit `make_move`). Aufgrund der Tatsache, dass dieses neue Board eine Spielposition, also einen Node darstellt, ist diese Struktur nicht mehr notwendig.

Innerhalb von `find_best_move/6` wird die Funktion `negamax/5` aufgerufen, die der Beschreibung aus dem Kapitel 2.2, mit Alpha-Beta Optimierung folgt. Der Abbruch der Schleife wird durch Punkt 4a im Ablaufdiagramm vollzogen und wie folgt realisiert:

```
% 4e Prüfe Alpha >= Beta (Pruning)
case get_alpha(NewAlphaBeta) >= get_beta(NewAlphaBeta) of
true ->
% Pruning: Abbruch, bester Zug gefunden
{ok, NewBestMove};
false ->
% Fahre mit nächstem Zug fort. Der Rekursive aufruf
find_best_move(Board, Rest, Player, NewAlphaBeta, NewBestMove, NewBestValue)
end.
```

Wenn diese Abbruchbedingung erreicht ist, also alle Züge bewertet wurden oder durch das Pruning die Funktion endet, wird der Zug auf dem Board mittels `execute_move/2` ausgeführt.

## 5 Analyse

Die Implementierung in `bot_Leonhard_Luedeke.erl` bildet den Ablauf des Ablaufdiagrammes 7.1.1 (Punkte 0-6) direkt nach. Mit `make_move/1` (Z. 21) wird der Bot gestartet und mittels `check_if_running/1` (Z. 75) der Spielstatus geprüft. Anschließend wird der aktuelle Spieler via `get_current_player/1` (Z. 83) sowie die möglichen Züge über `get_possible_moves/1` (Z. 90) und `has_moves/1` (Z. 93) ermittelt.

Die Zugreihenfolge wird mit `sort_moves_by_relevance/1` (Z. 100) vorbereitet und das Suchfenster über `create_alphabeta/2` (Z. 64) initialisiert. Die Suche wird in `find_best_move/6` (Z. 116) ausgeführt und den finalen Zug mittels `execute_move/2` (Z. 305) an `connect4:make_move/2` delegiert.

Der relevanteste Bezug zum Entwurf und Code in `find_best_move/6` (Z. 116) sichtbar, da der Schritt 4a über die Simulation eines Nachfolgezustands (per `connect4:make_move/2` in Z. 125) erfolgt und die Schritte 4b-4d durch den Aufruf von `negamax/5` (Z. 155) mit konsistenter Negation des Suchfensters (via `get_beta/1` in Z. 68, `get_alpha/1` in Z. 67, `negate/1` in Z. 335, `negate_player/1` in Z. 337) abgebildet werden. Woraufhin die Aktualisierung des Besten Zuges ausschließlich über `is_better/2` (Z. 331) sowie die Anpassung von Alpha mittels `maximum/2` (Z. 328) und `set_alpha/2` (Z. 69) erfolgt und der Abbruchfall 4e genau der Bedingung `get_alpha(NewAlphaBeta) >= get_beta(NewAlphaBeta)` (Z. 149-154) entspricht.

Um die rekursiven Bewertungen möglichst überschaubar zu halten, sind im `negamax/5` (Z. 155) die Bewertung terminaler Zustände und Tiefenlimit getrennt (Terminalbewertung über `evaluate_terminal/3` in Z. 217 und Heuristik bei Tiefe 0 über `evaluate_board/2` in Z. 201). Außerdem wurde über `negamax_recursive/5` (Z. 170) die potentielle ermittelt und die Rekursive Iteration über Kindzüge in `negamax_loop/7` (Z. 179) realisiert. Dabei findet ein Alpha-Beta-Cutoff exakt bei `NewAlpha >= Beta` (Z. 190-195) statt und damit ausschließlich Äste verwirft, die das aktuelle Suchfenster nicht mehr verbessern können 2.2.

Der 4-Fenster-Score und der Center-Bonus sind mithilfe der Funktion `evaluate_board/2` (Z. 201) umgesetzt, die Board Funktion `connect4:export_board/1` nutzt um das Board als Zeilenweise darstellung zu exportieren. Anschließend wird `evaluate_rows/3` (Z. 224) mit `extract_windows/2` (Z. 258) sowie `score_windows/3` (Z. 264) und `score_window/2` (Z. 269) aufgerufen um eine Zeilenbasierte Bewertung (Zählfunktionen: `count_color/3` in Z. 285 und `count_empty/2` in Z. 293) zu vollziehen. Zusätzlich werden die Spalten über `evaluate_cols/3` (Z. 230) unter Nutzung von `get_column_from_rows/3` (Z. 238) berücksichtigt und schließlich den Center-Bonus über `evaluate_center_rows/2` (Z. 243) und `count_center_color/4` (Z. 247) auf die mittlere Spalte abbildet.

Die Angesprochene Relevanz der Zugreihenfolge für die Effektivität des Alpha-Beta-Prunings 2.2, wird mit `sort_moves_by_relevance/1` (Z. 100) und einer Festen Reihenfolge `[4, 3, 5, 2, 6, 1, 7]` berücksichtigt um damit früh starke Züge zu evaluieren. Dies soll die Wahrscheinlichkeit erhöhen, dass sowohl in `find_best_move/6` (Z. 116) als auch in `negamax_loop/7` (Z. 179) eine frühzeitige Beendigung der Suche eintreten.

## 5.1 Tournaments

Da die Ergebnisse des Bots sehr davon abhängen wie die Konstanten gewählt sind werde ich hier 3 verschiedene Tournaments mit verschiedenen Konstanten auflisten. Der Bot fängt in diesen Szenarien immer an.

### 5.1.1 1. Versuch

Hierbei sei gesagt das alle Spiele immer exakt gleich ausgehen. Das ist auch nicht überraschend, da es keine Zufallskomponente gibt die hier variation in die Bewertung einbringt. Dies gilt auch die Nachfolgenden Versuche.

```
-define(WIN, 10000) .
-define(LOSE, -10000) .
-define(THREE_IN_ROW, 100) .
-define(TWO_IN_ROW, 10) .
-define(CENTER_BONUS, 5) .
-define(INITIAL_ALPHA, -1000000) .
-define(INITIAL_BETA, 1000000) .
-define(SEARCH_DEPTH, 6) .
-define(BOARD_WIDTH, 7) .
-define(BOARD_HEIGHT, 6) .
-----
Ergebnis:
{Blue,Red,Remis,Moves}
{0      , 50,      0,    550}
```

Somit brauch der Bot bei diesen Konstanten 11 Züge bis zum Sieg. Doch wie sieht es aus wenn wir die Suchtiefe verändern? Dazu betrachten wir Versuch 2.

|       |       |       |   |       |       |       |
|-------|-------|-------|---|-------|-------|-------|
| empty | empty | empty | B | empty | empty | empty |
| empty | empty | empty | R | empty | empty | empty |
| empty | empty | empty | B | empty | empty | empty |
| empty | empty | B     | R | empty | empty | empty |
| empty | empty | B     | B | empty | empty | empty |
| empty | R     | R     | R | R     | empty | empty |

Abbildung 4: Endstellung 1. Versuch

### 5.1.2 2. Versuch

Generell erwarte ich, dass die Berechnungen schneller gehen da die Rekursion nicht so weit absteigen muss. Jedoch würde ich ab einer zu niedrigen Suchtiefe erwarten das der Bot evt. mehr Züge bis zum gewinnen benötigt. Die Ergebnisse bei einer Schrittweise Verringerung der Suchtiefe sehen wie folgt aus:

Tabelle 1: Versuchsergebnisse bei reduzierung der Suchtiefe

| SEARCH_DEPTH | Züge im Schnitt | WIN or LOSE | Anzahl Games |
|--------------|-----------------|-------------|--------------|
| 6            | 11              | WIN         | 50           |
| 5            | 7               | WIN         | 50           |
| 4            | 11              | WIN         | 50           |
| 3            | 7               | WIN         | 50           |
| 2            | 25              | WIN         | 50           |
| 1            | 7               | WIN         | 50           |

Hier wurde Erwartung nicht erfüllt. Die Verringerung Suchtiefe hat zwar Einfluss auf Züge die es bis zum Gewinnen braucht aber nicht unbedingt negativ.

### 5.1.3 3. Versuch

Nachdem wir sehen können das die Suchtiefe einen Einfluss darauf hat wie viele Züge der Bot zum Sieg braucht könnte es auch Interessant sein wie sehr der Center Bonus einen Einfluss darauf hat. Hier die Ergebnisse einer Schrittweise Verringerung des CENTER\_BONUS bei gleichbleibender Suchtiefe von 6.

Tabelle 2: Versuchsergebnisse bei reduzierung der Suchtiefe

| SEARCH_DEPTH | CENTER_BONUS | Züge im Schnitt | WIN or LOSE | Anzahl Games |
|--------------|--------------|-----------------|-------------|--------------|
| 6            | 5            | 11              | WIN         | 10           |
| 6            | 4            | 11              | WIN         | 10           |
| 6            | 3            | 11              | WIN         | 10           |
| 6            | 2            | 11              | WIN         | 10           |
| 6            | 1            | 11              | WIN         | 10           |
| 6            | 0            | 11              | WIN         | 10           |

Es hat den Anschein, als ob der Center Bonus gegen den rudimenteren Bot keinen Einfluss hat.

## 6 Fazit

Rückblickend zeigt die Arbeit das sich die gelieferte Schnittstelle gut mit dem im gewählten Ansatz implementieren lässt. Außerdem hat mir diese Hausarbeit die Funktionale Programmierung sehr viel näher gebracht und besonders die Erkenntnis, die Anfangs gewählte Node Struktur nicht zu benötigen hat mir gezeigt welche Vorteile diese Art der Programmierung bringen kann. Jedoch viel es nicht immer einfach aus dem Klassischen Objektorientierten Programmierparadigma auszubrechen.

Die Versuche zeigten das die Implementierung funktioniert und den Bot immer schlägt und zeigt auf das die Suchtiefe des Algorithmuses einen Einfluss auf die Spielstärke hat. Wohingegen der Center-Bonus keinen erkennbaren Einfluss auf das Ergebnis hat.

Um bessere Aussagen über die Effektivität des Bots treffen zu können würde sich nun weitere Tournaments gegen stärkere Bots anbieten. Dabei können die Konstanten angepasst werden um zu ergründen welche Konstanten welchen Einfluss auf das Spiel nehmen.

## 7 Anhang

### 7.1 Datenstrukturen

#### Board (Hauptdatenstruktur)

`Board = {board, Matrix, CurrentPlayer, Score}`

- `Matrix`: Liste von 7 Spalten (Spaltenweise Organisation aufgrund der Spielmechanik.)
- `CurrentPlayer`: `{Color, Count}` mit `Color`  $\in$  `{red, blue}`, `Count`  $\geq 0$
- `Score`: `running | red | blue | remis`

#### Spalte

`Column = [Cell1, Cell2, ..., Cell6]`

- Liste von 6 Zellen (von unten nach oben)
- `Cell`: `empty | {Color, Count}`

#### Koordinatensystem

- Links unten: (1,1), Rechts oben: (7,6)
- Spalten: 1–7 (horizontal), Reihen: 1–6 (vertikal)

#### 7.1.1 Wichtigsten Funktionen

##### `new_board/0` $\rightarrow$ `Board` [E1]

Initialisiert das Spiel und bedarf daher einer Näheren Betrachtung.

- Erzeuge 7 leere Spalten mit je 6 Zellen (`empty`)
- Setze `{red, 0}` als Startspieler, `Score = running`

**make\_move/2 (Board, ColIdx) → Board | Error [E2]**

Stellt die Kernfunktionalität dar, führt Spielzüge aus und aktualisiert den Zustand.

1. Prüfe: `Score = running? → sonst {error, {game_over, Score}}`
2. Prüfe: `1 ≤ ColIdx ≤ 7 → sonst {error, column_outofrange}`
3. Finde erste leere Zelle in Spalte `ColIdx` (von unten)
  - Falls keine: `{error, column_full}`
4. Setze Chip `{CurrentPlayer}` in gefundene Zelle
5. Berechne neuen `Score` via `score/1`
6. Berechne `NextPlayer = {andere Farbe, Count+1}`
7. Gebe `{board, NewMatrix, NextPlayer, NewScore}` zurück

**score/1 (Board) → Status [E3]**

Diese Funktion berechnet welcher Spieler Gewonnen bzw. verloren hat und ist damit essentiell.

1. Prüfe `winner_check(Matrix, red) → true: red`
2. Sonst prüfe `winner_check(Matrix, blue) → true: blue`
3. Sonst prüfe `is_full(Matrix) → true: remis`
4. Sonst: `running`

**winner\_check(Matrix, Color) → Boolean [E3.1]**

Eine sehr wichtige Hilfsfunktion für `score/1` und prüft ob 4 Steine nebeneinander liegen in allen Richtungen:

- Horizontal: Iteriere über Reihen
- Vertikal: Iteriere über Spalten
- Diagonal  $\searrow$ : Alle Diagonalen von links-oben nach rechts-unten
- Diagonal  $\nearrow$ : Alle Diagonalen von links-unten nach rechts-oben

**score\_ChipsRow/2 (Board, Player) → NumberList [E4]**

Wie in der Schnittstelle beschrieben, wird hier berechnet, wie viele gleichfarbige Chips um das nächste Freie Feld herum liegen. Die Vorgehensweise:

Für jede Spalte  $i$  (1..7):

1. Falls Spalte voll:  $-1$  (Somit ist die nicht Spielbar)
2. Sonst: Bestimme Position (ReihenIndex,  $i$ ) des nächsten freien Feldes
3. Zähle in alle 8 Richtungen gleichfarbige Chips:
  - Links, Rechts, Oben, Unten, 4 Diagonalrichtungen
4. Summiere alle Nachbarschaften  $\rightarrow [\text{Count}_1, \dots, \text{Count}_7]$

**check\_geq\_three/2 (Board, Player) → NumberList [E5]**

Überprüft in welchen Spaltennummern sich drei Chips (diagonal, horizontal oder vertikal) des Spielers zum freien Feld angrenzen.

- Nutze `score_ChipsRow/2` zur Berechnung
- Filter Spalten mit  $\text{Count} \geq 3 \rightarrow [\text{ColIdx}_1, \text{ColIdx}_2, \dots]$

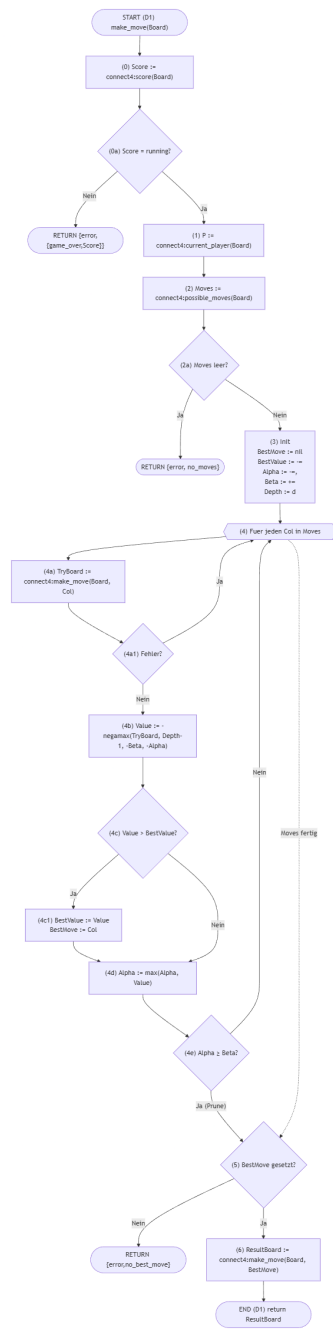


Abbildung 5: Ablaufdiagramm Zugentscheidung des Gamebots

## Literatur

- [1] *Connect 4 Algorithm*. – URL <https://roboticsproject.readthedocs.io/en/latest/ConnectFourAlgorithm.html>. – Zugriffsdatum: 2026-01-10. – Connect Four Robot documentation (Read the Docs)
- [2] ALLEN, James D.: Expert play in connect-four. In: *Verfügbar unter https://web.archive.org/web/20131023004851/http://homepages.cwi.nl/~tromp/c4.html* Allis 1988 (1990)
- [3] DENGEL, Andreas ; BERNS, Karsten ; BREUEL, Thomas M. ; BOMARIUS, Frank ; ROTH-BERGHOFER, Thomas R.: *KI 2008: Advances in Artificial Intelligence: 31st Annual German Conference on AI, KI 2008, Kaiserslautern, Germany, September 23-26, 2008, Proceedings*. Bd. 5243. Springer Science & Business Media, 2008
- [4] KNUTH, Donald E. ; MOORE, Ronald W.: An analysis of alpha-beta pruning. In: *Artificial Intelligence* 6 (1975), Nr. 4, S. 293–326. – URL <https://www.sciencedirect.com/science/article/pii/0004370275900193>. – ISSN 0004-3702
- [5] KOCSIS, Levente ; SZEPESVÁRI, Csaba: Bandit based Monte-Carlo Planning. In: *Proceedings of the European Conference on Machine Learning (ECML)*, Springer, 2006, S. 282–293
- [6] LIESE, Yassin: *Systematische Evaluierung und Optimierung Minimax-basierter Suchstrategien anhand des Spiels Vier-Gewinnt*. Februar 2024. – URL <https://doc.neuro.tu-berlin.de/bachelor/2024-BA-YassinLiese.pdf>
- [7] OWEN, Guillermo: *Spieltheorie*. Springer-Verlag, 2013
- [8] RUSSELL, Stuart J. 1. ; NORVIG, Peter 1. (Hrsg.): *Artificial intelligence a modern approach*. 3. ed., international ed. Boston Munich u.a. : Pearson, 2010 (Prentice-Hall series in artificial intelligence). – URL <http://www.gbv.de/dms/tib-ub-hannover/627596169.pdf>. – Literaturverz. S. 1063 - 1093
- [9] WIKIPEDIA: *Vier gewinnt* — *Wikipedia, The Free Encyclopedia*. <http://de.wikipedia.org/w/index.php?title=Vier%20gewinnt&oldid=260829991>. 2025. – [Online; accessed 26-November-2025]

## **Erklärung zur selbständigen Bearbeitung**

Hiermit versichere ich, dass ich die vorliegende Arbeit in allen Teilen selbstständig angefertigt und keine anderen als die in der Arbeit angegebenen Quellen und Hilfsmittel benutzt habe. Die in meiner Arbeit verwendeten KI-basierten Hilfsmittel habe ich (ggf. mit Produktnamen) angegeben.

Ich verantworte die Übernahme jeglicher von mir verwendeter maschinell generierter Passagen vollumfänglich selbst und trage die Verantwortung für eventuell durch die KI generierte fehlerhafte oder verzerrte Inhalte, fehlerhafte Referenzen, Verstöße gegen das Datenschutz- und Urheberrecht oder Plagiate.

Mir ist bewusst, dass wahrheitswidrige Angaben als Täuschungsversuch behandelt werden können.

---

Ort

---

Datum

---

Unterschrift im Original